7.3

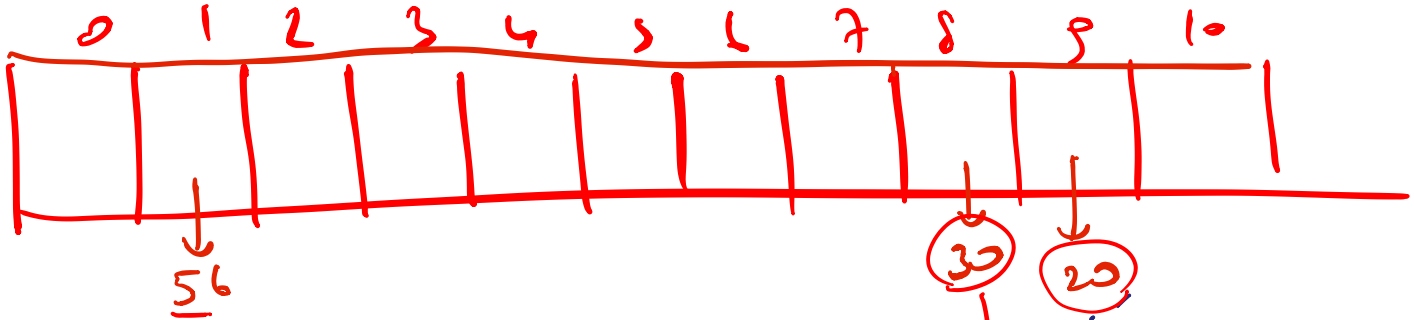
1. For the input 30, 20, 56, 75, 31, 19 and hash function $h(K) = K \bmod 11$

a. construct the open hash table.

b. find the largest number of key comparisons in a successful search in this table.

c. find the average number of key comparisons in a successful search in this table.

a)



30 mod 11 = 8

20 mod 11 = 9

56 mod 11 = 1

75 mod 11 = 9

31 mod 11 = 9

19 mod 11 = 8 largest key

19

75

31

b) for 31, # Comparison → 3

c) $\frac{1}{6} \times 1 + \frac{1}{6} \times 1 + \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 2 + \frac{1}{6} \cdot 3 + \frac{1}{6} \cdot 2$

$= \frac{1+1+1+2+3+2}{6} = \frac{10}{6} \approx 1.7$

7.3

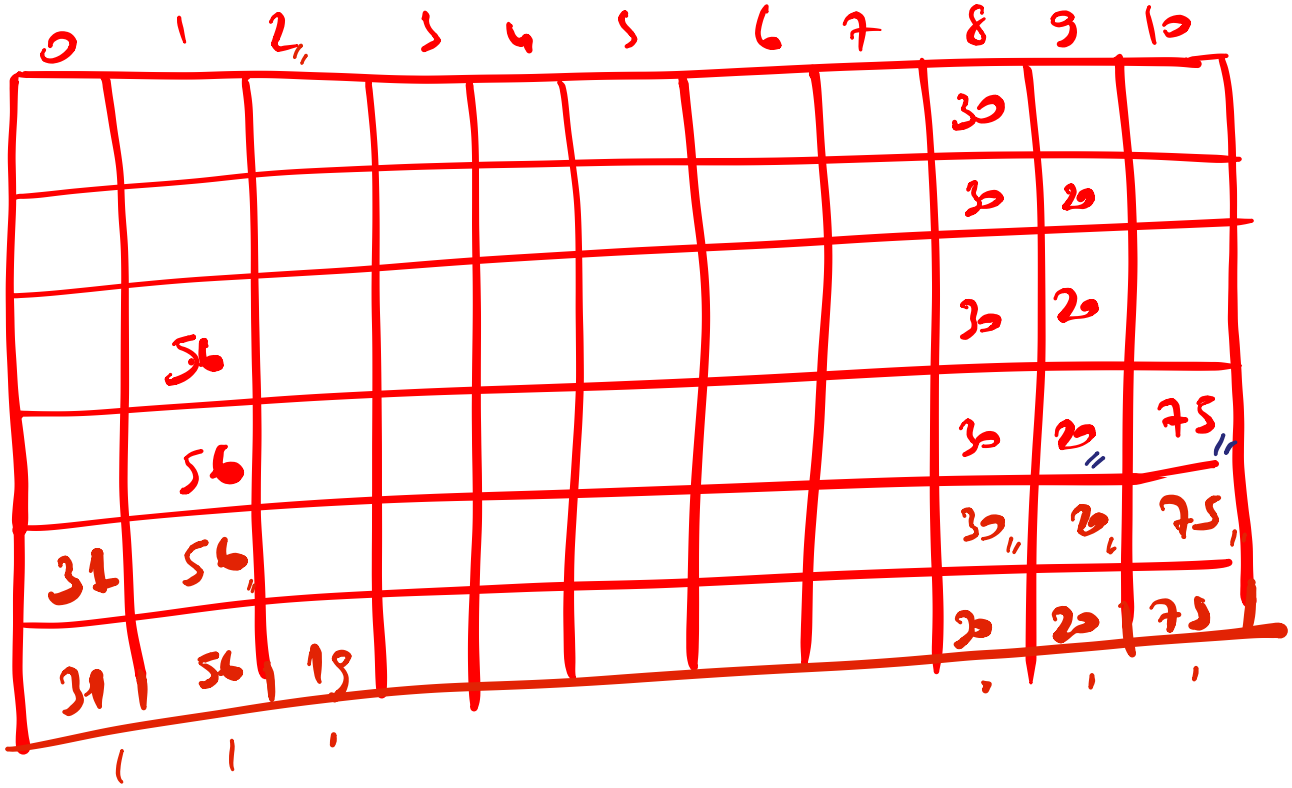
2. For the input 30, 20, 56, 75, 31, 19 and hash function $h(K) = K \bmod 11$

a. construct the closed hash table.

b. find the largest number of key comparisons in a successful search in this table.

c. find the average number of key comparisons in a successful search in this table.

a)



30 mod 11 = 8

20 mod 11 = 9

56 mod 11 = 1

75 mod 11 = 9

31 mod 11 = 9

19 mod 11 = 8

b) For 19, largest num. of key comp. = 6

c) $\frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 1 + \frac{1}{6} \cdot 2$

$\frac{1}{6} \cdot 3 + \frac{1}{6} \cdot 6 =$

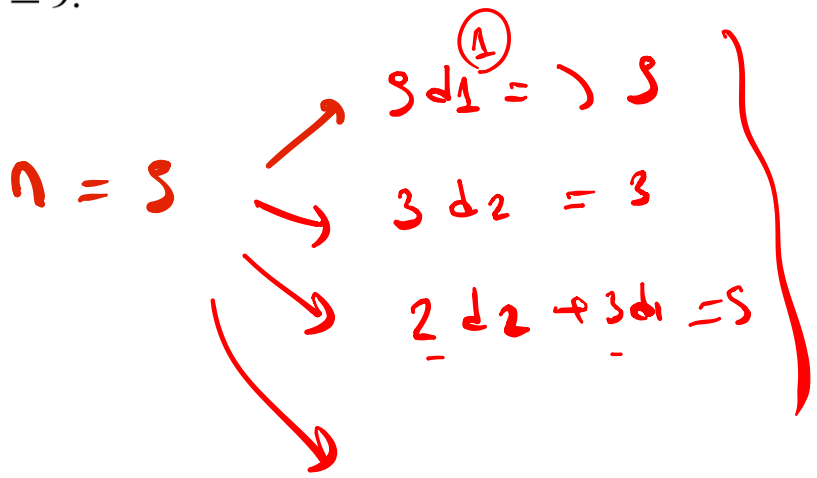
$= \frac{1+1+1+2+3+6}{6}$

$= \frac{14}{6} \approx 2.3$

8.1

4. Apply the dynamic programming algorithm to find all the solutions to the change-making problem for the denominations 1, 3, 5 and the amount $n = 9$.

d_1 d_2 d_3



F[n]: min # of coins to obtain given amount n.

F[0] = 0

F[n]: $\min_{n \geq d_j} \{1 + F[n - d_j]\}$ for $n > 0$

d_j ↓ denomination

→ 1, 3, 5

n	0	1	2	3	4	5	6	7	8	9
F	0	1	2	1	2	1	2	3	2	3

F[1] = $\min_{1 \leq n \geq d_j} \{F[1-1]+1\} = 1$

$F[2] = \min\{1 + F[2-1]\} = 2$

$F[3] = \min_{n \geq d_i}\{(1 + F[3-1]), (1 + F[3-3])\}$

$F[4] = \min\{(1 + F[4-1]), (1 + F[4-3])\}$

$F[5] = \min\{(1 + F[5-1]), (1 + F[5-3]), (1 + F[5-5])\}$

$F[6] = \min\{(1 + F[6-1]), (1 + F[6-3]), (1 + F[6-5])\}$

$$F[7] = \min_{7 \geq d_i} \{ (1 + F[7-1]), (1 + F[7-3]), (1 + F[7-5]) \}$$

$F[7-1] \to 2$, $1 + 2 = 3$

$F[7-3] \to 2$, $1 + 2 = 3$

$F[7-5] \to 2$, $1 + 2 = 3$

$$F[8] = \min_{8 \geq d_i} \{ (1 + F[8-1]), (1 + F[8-3]), (1 + F[8-5]) \}$$

$F[8-1] \to 3$, $1 + 3 = 4$

$F[8-3] \to 1$, $1 + 1 = 2$

$F[8-5] \to 1$, $1 + 1 = 2$

$$F[9] = \min_{9 \geq d_i \in \{1,3,5\}} \{ (1 + F[9-1]), (1 + F[9-3]), (1 + F[9-5]) \}$$

$F[9-1] \to 2$, $1 + 2 = 3$

$F[9-3] \to 2$, $1 + 2 = 3$

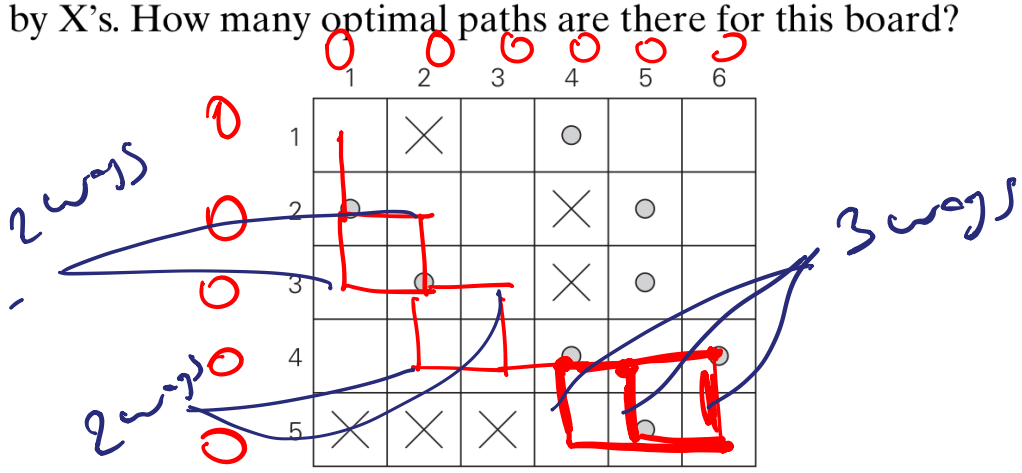
$F[9-5] \to 2$, $1 + 2 = 3$

coin sets $\to$ $1, 3, 5$

$\to$ $3, 3, 3$

8.1

5. How would you modify the dynamic programming algorithm for the coin-collecting problem if some cells on the board are inaccessible for the robot? Apply your algorithm to the board below, where the inaccessible cells are shown by X's. How many optimal paths are there for this board?



$F[i,j]$ = Largest num. of coins collected by the robot and bring to cell (i,j)

$F[0,j] = 0$, for $1 \le j \le n$

$F[i,0] = 0$, for $1 \le i \le m$

$c_{ij} = 0$ if there is no coin in cell (i,j)

$c_{ij} = 1$ if there is a coin in cell (i,j)

$$F[i,j] = c_{ij} + \max\{F[i-1,j], F[i,j-1]\}$$

for $1 \le i \le m$, $1 \le j \le n$

$F[1,1] = \underbrace{c_{11}}_{0} + \max\{\underbrace{F[0,1]}_{0}, \underbrace{F[1,0]}_{0}\} = 0$

$F[2,1] = \underbrace{c_{21}}_{1} + \max\{\underbrace{F[1,1]}_{0}, \underbrace{F[2,0]}_{0}\}$

$F[2,1] = 1$

$F[2,2] = \underbrace{c_{22}}_{0} + \max\{\overset{X}{\underbrace{F[1,2]}_{X}}, \overset{1}{\underbrace{F[2,1]}_{1}}\} = 1$

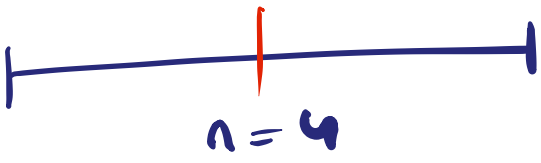
$F[5,6] = \max\{\underbrace{F[4,6]}_{4}, \underbrace{F[5,5]}_{4}\} + \underbrace{c_{56}}_{0} = 4$



$2 \cdot 2 \cdot 3 = 12$ ways

8.1

6. *Rod-cutting problem* Design a dynamic programming algorithm for the following problem. Find the maximum total sale price that can be obtained by cutting a rod of n units long into integer-length pieces if the sale price of a piece i units long is p_i for $i = 1, 2, \ldots, n$. What are the time and space efficiencies of your algorithm?



length of the rod

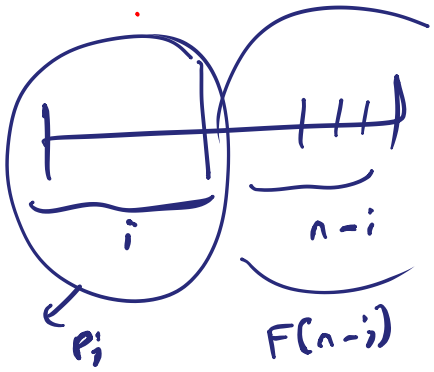
n	1	2	3	4
p_i	1	5	8	9

p_1 p_2 p_3 p_4

price of a cut i

cut	price
1, 3	1 + 8 = 9
2, 2	5 + 5 = 10
1, 1, 2	1 + 1 + 5 = 7
4	9

a, b, c, d — max



F(n) : maximum price

F[0] = 0

$$F(n) = \max_{1 \le i \le n} \{ p_i + F[n-i] \} \quad \text{for } n > 0$$

For n = 4

- $F[1] = \max\{ p_1 + F[1-1] \} = \max\{ \underset{1}{p_1} + \underset{0}{F[1-1]} \} = 1$
- $F[2] = \max_{1 \le i \le 2} \{ (p_1 + F[2-1]), (p_2 + F[2-2]) \} = 5$

$p_1 = 1$, $F[2-1] = 1$, $p_2 = 5$, $F[2-2] = 0$, 5

$$F[3] = \max_{1 \le i \le 3} \{ (P_1 + F[3-1], P_2 + F[3-2], P_3 + F[3-3] \}$$

P_1: 1, $F[3-1]$: 5 → 6

P_2: 5, $F[3-2]$: 1 → 6

P_3: 8, $F[3-3]$: 0 → 8

$= 8$

$$F[4] = \max_{1 \le i \le 4} \{ P_1 + F[4-1], P_2 + F[4-2], P_3 + F[4-3] + P_4 + F[4-4] \}$$

$P_1 + F[4-1]$: 5

$P_2 + F[4-2]$: 10

$P_3 + F[4-3] + P_4 + F[4-4]$: 9

$= 10$

$P_2 \rightarrow F[2] \rightarrow P2 \Rightarrow 2, 2$

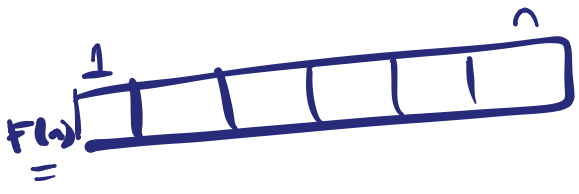
Time Complexity

→ F[1] → 1
F[2] → 2
F[3] → 3
⋮
F[n] → n

$1 + 2 + 3 + \cdots + n = n^2$

$\frac{n(n+1)}{2} \rightarrow n^2 \rightarrow$ Time Complexity

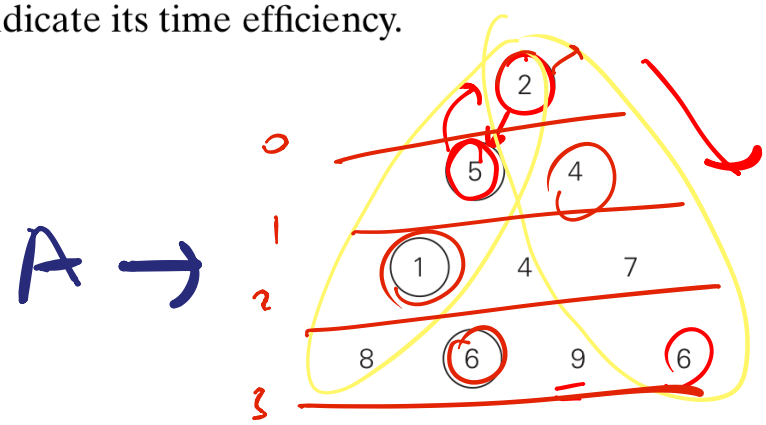
$T(n) = \Theta(n^2)$



Space Complexity : $\Theta(n)$

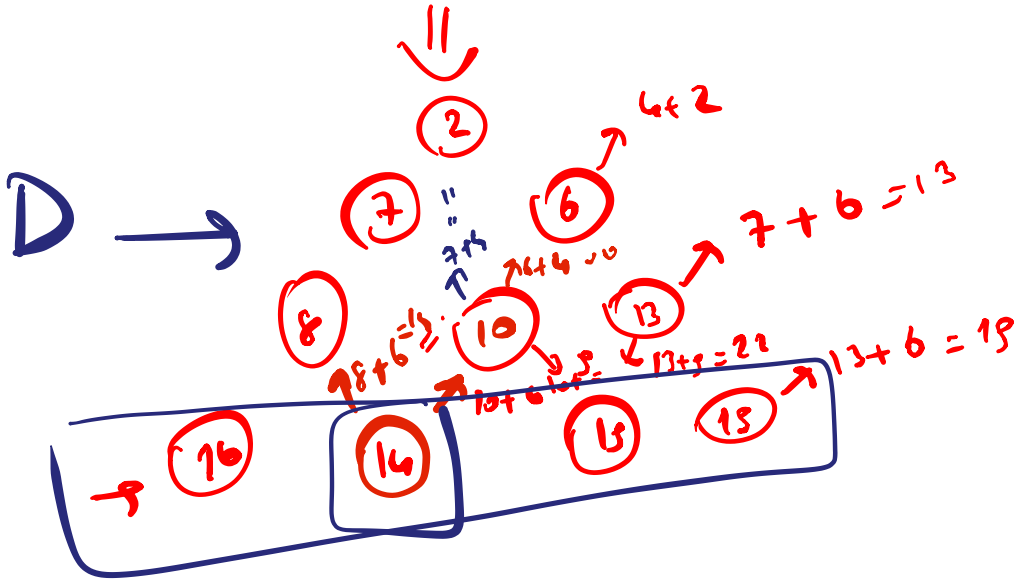
8.1

8. *Minimum-sum descent* Some positive integers are arranged in an equilateral triangle with n numbers in its base like the one shown in the figure below for $n = 4$. The problem is to find the smallest sum in a descent from the triangle apex to its base through a sequence of adjacent numbers (shown in the figure by the circles). Design a dynamic programming algorithm for this problem and indicate its time efficiency.



Space → n

Time Complexity : n^2



Dynamic Approach

- Build a solution by moving from the apex toward the base.
- Calculate minimum cumulative sum for each cell (i, j)

- $D[:][j]$: minimum path sum to reach that cell.

- $A[:][j]$: value in the cell (i,j) in the triangle A.

- for apex: $D[0][0] = A[0][0]$

- Left edges $(j = 0)$

$$D[i][0] = A[:][0] + D[:-1][0]$$

- Right Edges $(j = i)$

$$D[:][i] = A[:][:] + D[:-1][:-1]$$

- Internal cells $(0 < j < i)$

$$D[i][j] = A[:][j] + \min(D[i-1][j-1], D[i-1][j])$$

- After all sums computed, find the smallest element in the base of the triangle D.

8.1

1. a. Apply the bottom-up dynamic programming algorithm to the following instance of the knapsack problem:

item	weight	value
1	3	$25
2	2	$20
3	1	$15
4	4	$40
5	5	$50

capacity $W = 6$.

b. How many different optimal subsets does the instance of part (a) have?

c. In general, how can we use the table generated by the dynamic programming algorithm to tell whether there is more than one optimal subset for the knapsack problem's instance?

items i \ capacity j	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	0	0	25	25	25	25
2	0	0	20	25	25	45	45
3	0	15	20	35	40	45	60
4	0	15	20	35	40	55	65
5	0	15	20	35	40	55	65

V[1,1]=0
V[1,2]=0
V[1,3]=25
V=[1,4]=25
V=[1,5]=

V[i,j] = optimal value of such instance

V[0,j] = 0
V[i,0] = 0

$V[i,j] = \max\{V[i-1,j], v_i + V[i-1, j-w_i]\}$ if $j \geq w_i$

$V[i,j] = V[i-1,j]$ for $j - w_i < 0$

max. value of feasible set $= V[5,6] = 65$, optimal solution item 3, item 5